Algorithm Design: Backtracking Solutions

Name: S. Jaxon Rizal | **Reg No:** 192511421

Code of Conduct:
*I, **S. Jaxon Rizal / 192511421** (Name / Reg No), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *S. Jaxon Rizal*

Q1. N-Queens Problem

Inputs: Board size N (integer).

Output: All valid queen placements where no two queens attack each other.

Algorithm Strategy:

Place queens column by column. For each column, try placing a queen in every row. Validate the placement by checking the current row and both diagonals. If valid, recursively proceed to the next column. If placing a queen leads to no solution, backtrack by removing it.

Pseudocode:

```
FUNCTION SolveNQueens(N)
    board = NxN matrix initialized to 0
    results = empty list
    PlaceQueen(board, 0, N, results)
    RETURN results

FUNCTION PlaceQueen(board, col, N, results)
    IF col >= N THEN
        ADD copy of board to results
        RETURN

    FOR row FROM 0 TO N - 1 DO
        IF IsSafe(board, row, col, N) THEN
            board[row][col] = 1       // Place queen
            PlaceQueen(board, col + 1, N, results) // Recurse
            board[row][col] = 0       // Backtrack
        END IF
    END FOR

FUNCTION IsSafe(board, row, col, N)
    // Check left side of current row
    FOR i FROM 0 TO col - 1 DO
        IF board[row][i] == 1 THEN RETURN FALSE

    // Check upper diagonal on left
    FOR i, j = row, col; i >= 0 AND j >= 0; i--, j-- DO
        IF board[i][j] == 1 THEN RETURN FALSE

    // Check lower diagonal on left
    FOR i, j = row, col; i < N AND j >= 0; i++, j-- DO
        IF board[i][j] == 1 THEN RETURN FALSE

    RETURN TRUE
```

Q2. Subset Sum Problem

Inputs: Array of integers S, target sum T.

Output: True if a subset exists that sums to T, False otherwise (and optionally output the subset).

Algorithm Strategy:

Explore all subsets using include/exclude decisions. Keep track of the current sum. If the sum matches the target, record success. Prune the search tree if the current sum exceeds the target (assuming positive integers).

Pseudocode:

```
FUNCTION SubsetSum(S, target)
    current_subset = empty list
    RETURN FindSubset(S, target, 0, 0, current_subset)

FUNCTION FindSubset(S, target, index, current_sum, current_subset)
    IF current_sum == target THEN
        PRINT current_subset
        RETURN TRUE

    // Pruning
    IF current_sum > target OR index >= length(S) THEN
        RETURN FALSE

    // Include current element
    current_subset.append(S[index])
    IF FindSubset(S, target, index + 1, current_sum + S[index], current_subset)
THEN
        RETURN TRUE

    // Exclude current element (Backtrack)
    current_subset.pop()
    IF FindSubset(S, target, index + 1, current_sum, current_subset) THEN
        RETURN TRUE

    RETURN FALSE
```

Q3. Graph Coloring Problem

Inputs: Graph $G(V, E)$ represented as an adjacency matrix, number of colors M.

Output: Array mapping each vertex to a valid color, or failure if impossible.

Algorithm Strategy:

Iterate through vertices sequentially. For each vertex, try assigning colors 1 to M. Before assigning, check if any adjacent vertex shares the same color. If safe, assign and recurse for the next vertex. If no color works, backtrack.

Pseudocode:

```
FUNCTION SolveGraphColoring(graph, M)
    V = number of vertices in graph
    colors = array of size V initialized to 0
    IF AssignColor(graph, M, colors, 0, V) THEN
        RETURN colors
    ELSE
        RETURN "No Solution"

FUNCTION AssignColor(graph, M, colors, vertex, V)
    IF vertex == V THEN
        RETURN TRUE  // All vertices colored

    FOR color FROM 1 TO M DO
        IF IsSafeColor(vertex, graph, colors, color, V) THEN
            colors[vertex] = color

            // Recurse for next vertex
            IF AssignColor(graph, M, colors, vertex + 1, V) THEN
                RETURN TRUE

            colors[vertex] = 0 // Backtrack
        END IF
    END FOR

    RETURN FALSE

FUNCTION IsSafeColor(vertex, graph, colors, color, V)
    FOR i FROM 0 TO V - 1 DO
        IF graph[vertex][i] == 1 AND colors[i] == color THEN
            RETURN FALSE
    RETURN TRUE
```

Q4. Hamiltonian Cycle Problem

Inputs: Graph $G(V, E)$ represented as an adjacency matrix.

Output: A valid sequence of vertices forming a Hamiltonian Cycle, or failure.

Algorithm Strategy:

Start at a fixed vertex (e.g., vertex 0). Attempt to add vertices one by one to the path. A vertex can be added if it is adjacent to the previously added vertex and not already in the path. Once all vertices are in the path, verify if the last vertex connects back to the start.

Pseudocode:

```
FUNCTION FindHamiltonianCycle(graph)
    V = number of vertices
    path = array of size V initialized to -1
    path[0] = 0  // Start at vertex 0

    IF ConstructPath(graph, path, 1, V) THEN
        PRINT path + [path[0]] // Print cycle
        RETURN TRUE
    ELSE
        RETURN "No Solution"

FUNCTION ConstructPath(graph, path, pos, V)
    // Base Case: All vertices included
    IF pos == V THEN
        // Check if last vertex is adjacent to start
        IF graph[path[pos - 1]][path[0]] == 1 THEN
            RETURN TRUE
        ELSE
            RETURN FALSE

    FOR vertex FROM 1 TO V - 1 DO
        IF IsValidVertex(vertex, graph, path, pos) THEN
            path[pos] = vertex

            IF ConstructPath(graph, path, pos + 1, V) THEN
                RETURN TRUE

            path[pos] = -1 // Backtrack
        END IF
    END FOR

    RETURN FALSE

FUNCTION IsValidVertex(vertex, graph, path, pos)
    // Check adjacency to previous vertex
    IF graph[path[pos - 1]][vertex] == 0 THEN
        RETURN FALSE

    // Check if vertex already in path
    FOR i FROM 0 TO pos - 1 DO
        IF path[i] == vertex THEN
            RETURN FALSE

    RETURN TRUE
```

Q5. Sudoku Solver

Inputs: A 9x9 grid where empty cells are 0.

Output: Completed 9x9 grid satisfying all Sudoku constraints.

Algorithm Strategy:

Find the first empty cell. Try assigning numbers 1-9 to the cell. Ensure the chosen number doesn't violate row, column, or 3x3 subgrid uniqueness. If valid, recursively solve the rest of the board. Backtrack if a dead end is reached.

Pseudocode:

```
FUNCTION SolveSudoku(grid)
    row, col = FindEmptyCell(grid)

    // No empty cells left, puzzle solved
    IF row == -1 AND col == -1 THEN
        RETURN TRUE

    FOR num FROM 1 TO 9 DO
        IF IsSafeSudoku(grid, row, col, num) THEN
            grid[row][col] = num

            IF SolveSudoku(grid) THEN
                RETURN TRUE

            grid[row][col] = 0 // Backtrack
        END IF
    END FOR

    RETURN FALSE

FUNCTION IsSafeSudoku(grid, row, col, num)
    // Check row constraint
    FOR x FROM 0 TO 8 DO
        IF grid[row][x] == num THEN RETURN FALSE

    // Check column constraint
    FOR x FROM 0 TO 8 DO
        IF grid[x][col] == num THEN RETURN FALSE

    // Check 3x3 subgrid constraint
    startRow = row - (row MOD 3)
    startCol = col - (col MOD 3)
    FOR i FROM 0 TO 2 DO
        FOR j FROM 0 TO 2 DO
            IF grid[i + startRow][j + startCol] == num THEN RETURN FALSE

    RETURN TRUE
```

Q6. Knight's Tour Problem

Inputs: Board size N.

Output: An NxN matrix showing the sequence of moves where the knight visits every square exactly once.

Algorithm Strategy:

Start from cell (0,0). Attempt to make one of 8 valid knight moves. Ensure the move stays within bounds and lands on an unvisited cell. Mark the cell with the current move number and recurse. Backtrack if no valid moves remain and the board is incomplete.

Pseudocode:

```
FUNCTION KnightsTour(N)
    board = NxN matrix initialized to -1
    // 8 possible moves for a knight
    moveX = [2, 1, -1, -2, -2, -1, 1, 2]
    moveY = [1, 2, 2, 1, -1, -2, -2, -1]

    board[0][0] = 0  // Starting step

    IF MoveKnight(board, 0, 0, 1, moveX, moveY, N) THEN
        RETURN board
    ELSE
        RETURN "No Solution"

FUNCTION MoveKnight(board, x, y, step, moveX, moveY, N)
    IF step == N * N THEN
        RETURN TRUE  // All cells visited

    FOR i FROM 0 TO 7 DO
        nextX = x + moveX[i]
        nextY = y + moveY[i]

        IF IsSafeKnight(nextX, nextY, board, N) THEN
            board[nextX][nextY] = step

            IF MoveKnight(board, nextX, nextY, step + 1, moveX, moveY, N) THEN
                RETURN TRUE

            board[nextX][nextY] = -1 // Backtrack
        END IF
    END FOR

    RETURN FALSE

FUNCTION IsSafeKnight(x, y, board, N)
    RETURN (x >= 0 AND x < N AND y >= 0 AND y < N AND board[x][y] == -1)
```

Q7. Permutation Generation Problem

Inputs: A set (or array) of elements A.

Output: A list of all unique arrangements (permutations) of the set.

Algorithm Strategy:

Fix elements one by one using a swap-based backtracking approach. At each recursion level representing an index, iterate over remaining elements, swap to fix the current position, recurse for the next index, and swap back to restore state (backtrack).

Pseudocode:

```
FUNCTION GeneratePermutations(A)
    results = empty list
    Permute(A, 0, length(A) - 1, results)
    RETURN results

FUNCTION Permute(A, left, right, results)
    IF left == right THEN
        ADD copy of A to results
        RETURN

    FOR i FROM left TO right DO
        Swap(A[left], A[i])
        Permute(A, left + 1, right, results) // Recurse
        Swap(A[left], A[i])                  // Backtrack
    END FOR
```

Q8. Maze Solving Problem (Rat in a Maze)

Inputs: A binary matrix $maze$ where 1 is open path and 0 is an obstacle.

Output: A binary matrix showing the path from (0,0) to destination (N-1, N-1).

Algorithm Strategy:

Starting at (0,0), attempt to move Right or Down (and optionally Up/Left if needed). Mark the visited cells in a solution matrix. If a move leads to a dead end, backtrack by unmarking the cell.

Pseudocode:

```
FUNCTION SolveMaze(maze, N)
    sol = NxN matrix initialized to 0
    IF FindMazePath(maze, 0, 0, sol, N) THEN
        RETURN sol
    ELSE
        RETURN "No Path"

FUNCTION FindMazePath(maze, x, y, sol, N)
    // Reached destination
    IF x == N - 1 AND y == N - 1 AND maze[x][y] == 1 THEN
        sol[x][y] = 1
        RETURN TRUE

    IF IsSafeMaze(maze, x, y, N) THEN
        IF sol[x][y] == 1 THEN RETURN FALSE // Already visited

        sol[x][y] = 1 // Mark as part of solution path

        // Move Forward in X direction (Down)
        IF FindMazePath(maze, x + 1, y, sol, N) THEN RETURN TRUE

        // Move Forward in Y direction (Right)
        IF FindMazePath(maze, x, y + 1, sol, N) THEN RETURN TRUE

        // (Optional: add Up and Left moves if maze allows moving backwards)

        // Backtrack
        sol[x][y] = 0
        RETURN FALSE
    END IF

    RETURN FALSE

FUNCTION IsSafeMaze(maze, x, y, N)
    RETURN (x >= 0 AND x < N AND y >= 0 AND y < N AND maze[x][y] == 1)
```

Q9. Combination Sum Problem

Inputs: Array of candidate numbers C and target sum T.

Output: All unique combinations in C where candidates sum to T.

Algorithm Strategy:

Sort the array. Start building combinations by picking an element. To allow infinite re-use of candidates, recurse without incrementing the index. To avoid duplicates, skip identical adjacent elements when moving forward. Stop when sum exceeds target.

Pseudocode:

```
FUNCTION CombinationSum(C, target)
    Sort(C)
    results = empty list
    current_combo = empty list
    FindCombos(C, target, 0, 0, current_combo, results)
    RETURN results

FUNCTION FindCombos(C, target, index, current_sum, current_combo, results)
    IF current_sum == target THEN
        ADD copy of current_combo to results
        RETURN

    FOR i FROM index TO length(C) - 1 DO
        // Prune if sum exceeds target
        IF current_sum + C[i] > target THEN BREAK

        // Skip duplicates to avoid duplicate combinations
        IF i > index AND C[i] == C[i - 1] THEN CONTINUE

        current_combo.append(C[i])

        // Recurse (use i to allow reusing the same element, or i+1 for unique
usage)
        FindCombos(C, target, i, current_sum + C[i], current_combo, results)

        current_combo.pop() // Backtrack
    END FOR
```

Q10. Word Search Problem (Grid Search)

Inputs: 2D character grid ***board*** and a string ***word***.

Output: True if the word exists in the grid, False otherwise.

Algorithm Strategy:

Iterate through every cell in the grid to find a match for the first character of the word. Initiate a DFS/backtracking search exploring adjacent cells (up, down, left, right). Temporarily modify the current cell to prevent revisiting during the search, restoring it afterward.

Pseudocode:

```
FUNCTION WordSearch(board, word)
    rows = length(board)
    cols = length(board[0])

    FOR r FROM 0 TO rows - 1 DO
        FOR c FROM 0 TO cols - 1 DO
            IF board[r][c] == word[0] AND Search(board, word, r, c, 0) THEN
                RETURN TRUE
    RETURN FALSE

FUNCTION Search(board, word, r, c, index)
    IF index == length(word) THEN
        RETURN TRUE

    IF r < 0 OR r >= length(board) OR c < 0 OR c >= length(board[0])
       OR board[r][c] != word[index] THEN
        RETURN FALSE

    // Mark as visited by altering character
    temp = board[r][c]
    board[r][c] = '#'

    // Explore all 4 directions
    found = Search(board, word, r + 1, c, index + 1) OR
            Search(board, word, r - 1, c, index + 1) OR
            Search(board, word, r, c + 1, index + 1) OR
            Search(board, word, r, c - 1, index + 1)

    // Backtrack
    board[r][c] = temp

    RETURN found
```